

Goto Statement in C# with Examples

In this article, I am going to discuss **Goto Statement in C#** with Examples. Please read our previous articles, where we discussed [Continue Statement in C#](#) with Examples. At the end of this article, you will understand the goto Statement in C# and when and how to use the goto statement in the C# language with examples.

Goto Statement in C#

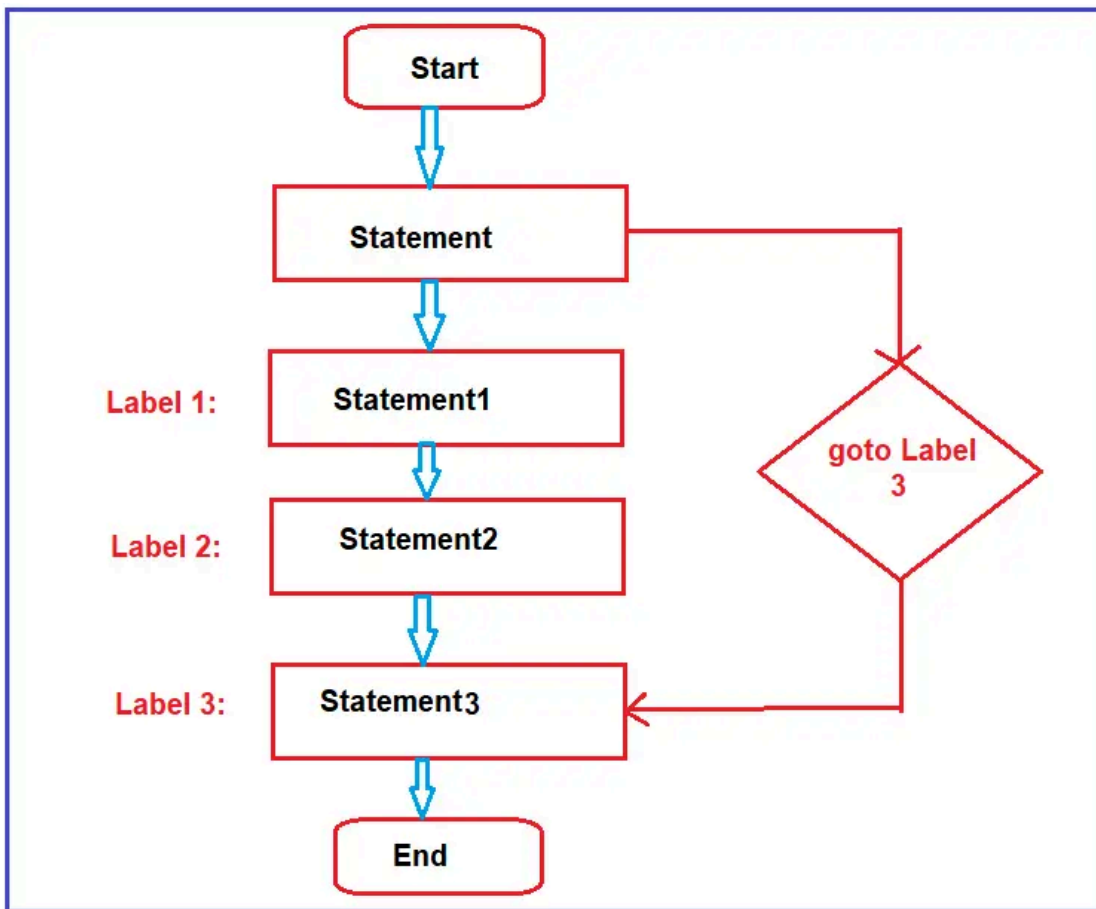
The Goto Statement in C# is used to transfer the control to the labeled statement in the program. The label is a valid identifier and placed just before the statement from where the control is transferred. That means the goto Statement provides an unconditional jump from the goto to a labeled statement in the same function.

Goto is a keyword and by using this goto keyword we can pass the control anywhere in the program in the local scope. When we are working with the goto statement it required an identifier called a label. Any valid identifier followed by a colon is called a label. Whenever we are working with a goto statement it is called an unstructured control flow statement because it breaks the rule of structure programming language.

The goto statement is rarely used because it makes the program confusing, less readable, and complex. Also, when it is used, the control of the program won't be easy to trace, hence it makes testing and debugging difficult.

Goto Statement Flowchart:

The following diagram shows the flowchart of the goto statement in C#. Here, as you can see in the below image, we have three labels i.e. Label 1, Label 2, and Label 3. Whenever we are executing our application code, if we have written goto label name, for example, goto Label 3, then the control will immediately jump to the statement which is written after the Label 3 identifier.



Syntax to use goto statement in C# Language:

```
goto label;  
..  
label: statement;
```

Example to Understand the goto statement in C# Language:

Let us understand the goto statement in C# with an example. Please have a look at the following code. Here, you can see that we have created a label (or identifier) called Label1 and inside the Main method after printing the 1st Statement, we call the Label1 by using the goto statement. In this case, the control will directly jump to the Label1 label and start executing from the 3rd Statement. In this case, the 2nd Statement will be executed.

```
using System;  
namespace JumpStatementDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Console.WriteLine("1st Statement: ");  
            goto Label1;  
        }  
    }  
}
```

```

        goto label1;    //goto label label1:
        Console.WriteLine("2nd Statement: ");
label1:    //label:
        Console.WriteLine("3rd Statement: ");
        Console.WriteLine("End of Main Method Statement: ");
        Console.ReadKey();
    }
}

```

Output:

```

1st Statement:
3rd Statement:
End of Main Method Statement:

```

It is also possible in C# that we can have multiple labels. When a label match is found, it will execute the statements as well as the other label statements which are appeared after this match label statements. For a better understanding, please have a look at the below example. Here, we have three labels i.e. Label1, Label2, and Label2. And we have written goto Label2, so, this will jump to Label2 and it will start executing the code from Label2 as well as it will execute the code written for Label3.

```

using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("1st Statement: ");
            goto label2;    //goto label label2:
            Console.WriteLine("2nd Statement: ");
label1:    //label:
            Console.WriteLine("3rd Statement: ");
label2:    //label:
            Console.WriteLine("4th Statement: ");
label3:    //label:
            Console.WriteLine("5th Statement: ");
            Console.WriteLine("End of Main Method Statement: ");
            Console.ReadKey();
        }
    }
}

```

Output:

```
1st Statement:  
4th Statement:  
5th Statement:  
End of Main Method Statement:
```

What happens if the label specified in the goto statement does not exist?

If you are specifying a label in the goto statement which does not exist in the current scope, then you will get a compiler error saying as shown in the below image.



Using goto statement inside a Loop in C#:

The goto statement transfers program control to a labeled statement. The label statement must exist in the scope of the goto statement. More than one goto statement can transfer control to the same label. This statement can be used to get out from a loop or an inner nested loop to an outer loop. For a better understanding, please have a look at the below example. In the below example, we are coming out from the loop using the goto statement. In this case, the statement written after the loop body and before the label statement will not be executed.

```
using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 1; i <= 10; i++)
            {
                if (i == 5)
                {

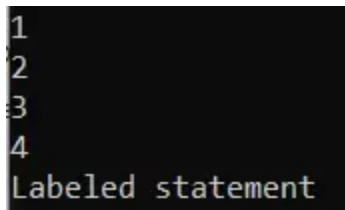
```

```

        goto Exitlabel;
    }
    Console.WriteLine(i);
}
Console.WriteLine("Next statement placed after loop");
Exitlabel: //goto label
    Console.WriteLine("Labeled statement");
    Console.ReadKey();
}
}
}

```

Output:



```

1
2
3
4
Labeled statement

```

Using goto in Switch Statement

Unlike the break statement, it does not transfer the program control to the next statement which is placed immediately after the loop or switch instead it will transfer the control to the statement written after the matched label.

You can also use the goto statement to transfer the control to a specific switch-case label or the default label in a switch statement. For a better understanding, please have a look at the below example. In the below example, in case 20, instead of break we have written goto case 5. So, in this case, it will transfer the control to case 5.

```

using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int number = 20;
            switch (number)
            {
                case 5:
                    Console.WriteLine("case 5");
                    break;
                case 10:
                    Console.WriteLine("case 10");
                    break;
                case 20:
                    Console.WriteLine("case 20");
                    goto case 5;
            }
        }
    }
}

```

```

        // goto statement transfer
        // the control to case 5
        goto case 5;
    default:
        Console.WriteLine("No match found");
        break;
    }
    Console.ReadKey();
}
}
}

```

Output:

```

case 20
case 5

```

Can we print numbers from 1 to 10 without using the loop in C#?

This is one of the frequently asked written interview questions. The interviewer will ask you to write a program to print the numbers from 1 to n without using the loop. It is possible in C# that without using a loop we can also achieve the loop functionality using the goto statement. For a better understanding, please have a look at the below example. In the below example, we are printing the numbers from 1 to 10 using the goto statement. In this case, each time it will check the count variable value and if it is less than or equals to 10, then it will execute the goto statement. When the count value becomes 11, the if condition becomes false and the goto statement will not execute.

```

using System;
namespace ControlFlowDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int count = 1;
label_Loop:
            Console.WriteLine(count);
            count++;
            if (count <= 10)
            {
                goto label_Loop;
            }
            Console.ReadKey();
        }
    }
}

```

Note: We should avoid using “goto Statement” wherever it is possible. It is very difficult to trace out the way of execution and figure out what the program is doing. and debugging and modifying such a program is very difficult. And all of our Programs Should consist of sequences, decisions, and loops.

Some Tricky Questions Related to C# goto statement

Question1: What will be the output of the below program?

```
using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.Write("C#Tutorials ");
            Console.Write("Welcome ");
            XYZ:
                Console.Write("X ");
                Console.Write("Y ");
                goto ABC;
                Console.Write("Programming ");
            ABC:
                Console.Write("Hello1 ");
                Console.Write("Hello2");
            Console.ReadKey();
        }
    }
}
```

Output: C#Tutorials Welcome X Y Hello1 Hello2

Note: In order to execute the program if the label has occurred it will be executed automatically without calling also. The creation of labels is always optional, after creating the label calling the label is also optional.

So, whenever we need to repeat the statement n number of times without using loops then we can use the goto statement but in the goto statement, we cannot place the break and continue statement.

Question2: What will be the output of the below program?

```
using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int i = 2;
            EVEN:
                Console.Write($"{i} ");
                i += 2;
                if (i <= 20)
                    goto EVEN;
        }
    }
}
```

```

        Console.ReadKey();
    }
}

```

Output: 2 4 6 8 10 12 14 16 18 20

Question3: What will be the output of the below program?

```

using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.Write("A");
            Console.Write("CTutorials");
            goto ABC;
            Console.Write("WELCOME");
            Console.Write("HELLO");
            abc:
                Console.Write("B");
                Console.Write("C");
            Console.ReadKey();
        }
    }
}

```

Output: **Error CS0159 No such label 'ABC' within the scope of the goto statement**

Note: In the goto statement, labels work with the help of case sensitivity i.e. upper case label and lower case label both are different.

In the next article, I am going to discuss [Functions in C#](#) with Examples. Here, in this article, I try to explain **Goto Statement in C# Language** with Examples. I hope you enjoy this Goto Statement in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.